INFORME DE ENTREGA PARA EL DESAFÍO I

Juan Pablo de los Ríos Cataño

Jorge Andrés Montoya Rendón

Facultad de Ingeniería, Universidad de Antioquía

2598521: Informática II

Aníbal José Guerra

20 de marzo de 2026

Resumen

En el presente informe, se busca retratar el proceso de solución para el desafío I propuesto en Informática II para el semestre 2026-1. Se espera retratar el análisis lógico de la solución, sus primeros planteamientos y la evolución de su estructura al cabo que se avanzaba en la solución del mismo. El informe, además, busca retratar la solución en sí misma, cómo funciona y por qué funciona; también se busca dar una retroalimentación objetiva de los resultados del desafío en los procesos de aprendizaje y al llegar a las conclusiones, una discusión de cómo la IA puede jugar un papel fundamental en este y por qué no debería considerarse una herramienta atípica o no apta para ejercicios evaluativos como este.

Palabras clave: desafío I, solución, evolución, estructura, IA.

Informe De Entrega Para El Desafío I

Cómo primeros pasos para el acercamiento a la solución del desafío I, es importante tener en cuenta su complejidad y empezar a retratar los primeros pensamientos acerca de posibles procesos lógicos o acercamientos a la solución; por esto mismo, cómo primer paso se planteó el "informe preliminar", en cuál se plantearon los primeros acercamientos a la solución y se fundamentó la estructura del trabajo. Es necesario, para entender el análisis y el problema como tal, tener la contextualización necesaria de lo que llamamos desafío I en Informática II.

Para 2026-1, en Informática II se dicta un curso de arquitectura de software, donde uno de sus principales objetivos es el de alcanzar un nivel alto en eficiencia en cuanto a programación se trata, para esto, el curso se dicta en el lenguaje de programación C++, con Qt cómo IDE de desarrollo; este último, se escogió a conveniencia por su facilidad con interfaces gráficas (algo muy requerido para el final del curso (UdeA). Teniendo esto en cuenta, es decir, el lenguaje de programación y los objetivos generales del curso, podemos plantear entonces el desafío I para 2026-1. El desafío I se basa en implementar una versión simplificada de tetris, que cuente con una implementación de manera modular, con interacciones directamente por consola, con restricciones en el uso de librerías cómo STL:

> La Standard Template Library (STL) es una colección de estructuras de datos genéricas y algoritmos escritos en C++. (Granada, s.f.)

Para esto, se debe usar operaciones a nivel de bits denominadas bitwise, donde resaltan máscaras, las cuales podemos definir como:

> El enmascaramiento de bits es una técnica de programación que permite realizar operaciones con datos binarios de forma más eficiente. Se emplea con frecuencia en algoritmos para mejorar el

> rendimiento en términos de complejidad temporal, utilizando operadores bit a bit para operaciones eficientes a nivel de bits. (Geeksforgeeks, 2023)

además, también es necesario el uso de memoria dinámica y, por ende, el uso de punteros, entre otros.

Primeros Acercamientos

Para lograr acercarse correctamente a la solución esperada, hay que entender primero cómo plantear el problema, por eso mismo, se establece una jerarquía "inicial" para saber que prioridad debe de tener cada parte de la solución a presentar, esta fue presentada en el informe preliminar, sin embargo, fue mutando mientras el proyecto avanzaba. De la solución final planteada, se generó la siguiente jerarquía:

Tablero.

Esta es la parte central del juego, es una de la que más validaciones necesita, además, de usar la estructura de datos más avanzada que se ha planteado hasta el momento en el curso, un puntero a punteros. Es por esto, que se considera la primera en jerarquía, puesto que es el corazón del proyecto, y en base a su implementación, se puede entender como ir implementando las demás partes del proyecto para que encajen en esta. Además, el detalle de que el usuario pueda ingresar el alto y el ancho deseado, deja aún más validaciones y adaptaciones a la estructura, para que sea compatible con la estructura del problema y de la solución a la que se espera llegar.

Piezas.

Si bien en la solución final la generación de las piezas parece un detalle menor, incluso repetitivo, en el planteamiento de la solución juega un factor clave en cómo ver el problema y

como llevar a cabo la solución. Sin piezas no hay tetris y su correcta generación ayuda incluso a simplificar el problema, por tanto, parece importante darle la atención que se merece a esta parte del problema y ser riguroso al momento de su implementación, teniendo en cuenta su compatibilidad con el tablero, que como se retrató anteriormente, es el eje central de la solución.

Juego.

Cuando se habla de juego en este contexto, se habla no solo de una parte del planteamiento del problema, también se menciona una parte fundamental de la solución; en el planteamiento final, igual que los mencionados anteriormente, juego aparece como un módulo de la entrega final, esto debido a su importancia dentro de la solución del problema. Aunque esto puede englobar muchas cosas, en este caso en particular, hablar de "juego" conlleva a hablar de un gran porcentaje de la lógica a nivel de bits que el desafío engloba; en "juego" se encuentran las operaciones que dictaminan el correcto funcionamiento de la solución planteada, se encuentra, aunque suena redundante, toda la lógica que rige el proyecto y que asegura su funcionamiento.

Validaciones.

Es probablemente el momento del proyecto que más cambios sufrió, porque si bien, en cuanto a importancia debería ser el primero, en cuanto a código y a modulación, fue el que más transformaciones sufrió respecto al informe preliminar. Si bien fue uno de los más fundamentales inicialmente, al avanzar en la solución del desafío, se fue volviendo cada vez más especifico para cada función empleada, y terminó transformándose en algo local, en algo más a discreción de cada parte del proyecto; además, de ser simplificada al implementar métodos como "switch case" o "try catch", entre otros.

Entradas.

En este módulo realmente se empieza a ver la parte final del proyecto, donde se interactúa con el usuario, donde se cumple el requisito de hacer la solución por medio de la terminal. En este se prioriza la claridad para el usuario final, para el "consumidor". También aprovecha bastante del elemento inmediatamente superior a este, es decir, las validaciones, pues donde más se esperan errores es en las partes que le corresponde al humano y no a la máquina. Además, dentro del código, es el módulo que menos complejidad requiere, pues se juega más que todo con evitar los errores humanos antes mencionados.

Implementación De La Solución

No es incorrecto, que cuando se empezó a hablar de jerarquía antes, se definieran, al menos de manera inicial, los módulos implementados para la solución; la estructura presentada anteriormente, coincide de hecho con los módulos de la solución dispuesta (a excepción de las validaciones como fue mencionado anteriormente), estos, coinciden con las principales necesidades y requerimientos del programa, estructurando, casi de la forma jerárquica en la que aquí fueron presentados, la solución que fue planteada. Es correcto entonces, luego de estas definiciones y de este ordenamiento, comenzar retratar la lógica a código de la solución encontrada para el desafío. Cabe resaltar, que, entre las aproximaciones de la solución, están incluidas, por ejemplo, el "camelCase" lo cual es "una convención muy usada en programación que usa la estructura que vemos en su nombre asimilando un camello" (MSMK | University College, 2025) para las variables, esto para poder leer las variables con facilidad y saber identificarlas.

Módulo Tablero.

En este, el principal reto, fue adaptar el tablero al mundo lógico, de manera tal que permitiera cambiarlo en tiempo de ejecución y realizar el juego. Para esto, se tomó la decisión de hacerlo dinámico, como un puntero a puntero, para maximizar la eficiencia en memoria y en operaciones al hacerse a nivel de bits. Para su creación, se implementó el siguiente algoritmo:

```
Algoritmo: creacionTablero (filas, columnas)

ENTRADA: filas (número de filas del tablero), columnas (número de
   columnas del tablero)

SALIDA: tablero (apuntador a apuntador que representa el tablero)

bytesPorFila = columnas / 8

tablero = reservar memoria para filas

PARA i = 0 HASTA filas - 1:

    tablero[i] = reservar memoria para bytesPorFila

    PARA j = 0 HASTA bytesPorFila - 1:

        tablero[i][j] = 0  // Inicializa cada byte en 0

    FIN PARA

FIN PARA

RETORNAR tablero

FIN Algoritmo
```

Cada bit de los bytes representa una columna: 0 = vacío, 1 = ocupado. La función devuelve un tablero completamente vacío. Además, al usar memoria dinámica, es importante garantizar su correcta liberación, esto para no tener una fuga de memoria y garantizar los estándares de entrega, eficiencia y optimización. Algoritmo de liberación:

```
Algoritmo: liberarMemoriaTablero (tablero, filas)

ENTRADA: tablero (apuntador a apuntador del tablero), filas (número de
   filas)

SALIDA: ninguna

PARA i = 0 HASTA filas - 1:

    liberar memoria de tablero[i]

FIN PARA

liberar memoria de tablero

FIN Algoritmo
```

Se recorre cada fila para liberar la memoria de los bytes y finalmente se libera la memoria del arreglo de filas. Esto asegura que no queden fugas de memoria después de usar el tablero. Cabe aclarar que no se mostrará todo el código en este o en cualquier otro módulo presentado, aquí se espera ilustrar el código más importante, la lógica que realmente sostiene la solución del problema y la hace posible.

Módulo Piezas.

En este, el reto inicial fue la creación de las piezas, reto, en el cuál se tuvieron varias aproximaciones antes de llagar a el planteamiento final, que es el “deseado” para la solución. De manera análoga al algoritmo presentado a continuación, se hicieron las 7 piezas que se usan en el juego:

```
Algoritmo: crearPiezaT ()

ENTRADA: ninguna

SALIDA: piezaT (apuntador al arreglo que representa la pieza T

piezaT = reservar memoria para 4 elementos

piezaT [0] = 0b111   // fila superior: tres bloques activos

piezaT [1] = 0b010   // fila del medio: bloque central activo

piezaT [2] = 0       // fila vacía

piezaT [3] = 0       // fila vacía

RETORNAR piezaT

FIN Algoritmo
```

Cada fila de la pieza se almacena como un byte, usando bits activos para representar los bloques. Esto permite manipular la pieza con operaciones “bitwise” y facilita rotaciones y detección de colisiones. Es conveniente hablar de la lógica de las rotaciones en este módulo, pues es uno de los ejes centrales del problema. La rotación inicia así:

```
Algoritmo: obtenerLimitesRotacion (pieza, filaMinima, filaMaxima,
columnaMinima, columnaMaxima)

ENTRADA: pieza (arreglo 4x4 de bits)

SALIDA: filaMinima, filaMaxima, columnaMinima, columnaMaxima

filaMinima = 4

filaMaxima = -1

columnaMinima = 4

columnaMaxima = -1

PARA fila = 0 HASTA 3:

    PARA columna = 0 HASTA 3:
```

```
            SI pieza[fila] tiene bit activo en columna:
                SI fila < filaMinima:
                    filaMinima = fila
                SI fila > filaMaxima:
                    filaMaxima = fila
                SI columna < columnaMinima:
                    columnaMinima = columna
                SI columna > columnaMaxima:
                    columnaMaxima = columna
                FIN SI
            FIN SI
        FIN PARA
    FIN PARA
    FIN Algoritmo
```

Determina las filas y columnas mínimas y máximas que contienen bloques activos de la pieza. Esto permite rotar la pieza trabajando únicamente sobre la región ocupada, ignorando espacios vacíos. Optimiza los cálculos y evita errores al rotar piezas dentro de la matriz de 4x4.

```
Algoritmo: rotarEnLimites (pieza, rotada, filaMinima, filaMaxima,
columnaMinima, columnaMaxima)
ENTRADA: pieza (arreglo original), rotada (arreglo destino), límites de
fila y columna
SALIDA: rotada (pieza rotada)
alto = filaMaxima - filaMinima + 1
PARA fila = filaMinima HASTA filaMaxima:
```

```
    PARA columna = columnaMinima HASTA columnaMaxima:
        SI pieza[fila] tiene bit activo en columna:
            nuevaFila = filaMinima + (columna - columnaMinima)
            nuevaColumna = columnaMinima + (alto - 1 - (fila -
filaMinima))
            activar bit en rotada[nuevaFila][nuevaColumna]
        FIN SI
    FIN PARA
FIN PARA
FIN Algoritmo
```

Rota la pieza 90° en sentido horario usando únicamente la región activa determinada por los límites. Calcula la nueva posición de cada bloque activo y lo ubica en el arreglo de destino. Esto asegura que la rotación sea correcta y que la pieza permanezca dentro de su espacio lógico.

```
Algoritmo: copiarPieza (origen, destino)
ENTRADA: origen (pieza original), destino (pieza destino)
SALIDA: destino (pieza copiada)
PARA i = 0 HASTA 3:
    destino[i] = origen[i]
FIN PARA
FIN Algoritmo
```

Copia los valores de una pieza a otra, evitando compartir la misma referencia en memoria. Esto es útil cuando se necesita manipular una pieza (por ejemplo, rotarla) sin modificar la original hasta que la rotación se confirme.

```
Algoritmo: rotacion(pieza)

ENTRADA: pieza (arreglo 4x4 de bits)

SALIDA: pieza (rotada)

rotada = arreglo de 4 elementos inicializados en 0

filaMinima, filaMaxima, columnaMinima, columnaMaxima = 0

LLAMAR obtenerLimitesRotacion (pieza, filaMinima, filaMaxima,
columnaMinima, columnaMaxima)

LLAMAR rotarEnLimites (pieza, rotada, filaMinima, filaMaxima,
columnaMinima, columnaMaxima)

LLAMAR copiarPieza (rotada, pieza)

FIN Algoritmo
```

Integra los pasos de rotación de una pieza: primero obtiene los límites activos, luego realiza la rotación dentro de esos límites, y finalmente actualiza la pieza original con la versión rotada. Este proceso modular permite manejar la rotación de forma eficiente y segura, evitando errores de desplazamiento. Además, de una función especial para la ubicación de las piezas de forma aleatoria.

```
Algoritmo: piezaAleatoria ()

ENTRADA: ninguna

SALIDA: pieza (apuntador a una pieza aleatoria)

random = número aleatorio entre 0 y 6

SI random == 0:

    RETORNAR crearT ()

SINO SI random == 1:

    RETORNAR crearJ ()
```

```
SINO SI random == 2:
    RETORNAR crearO ()
SINO SI random == 3:
    RETORNAR crearI ()
SINO SI random == 4:
    RETORNAR crearZ ()
SINO SI random == 5:
    RETORNAR crearS ()
SINO:
    RETORNAR crearL ()
FIN Algoritmo
```

Genera aleatoriamente una pieza entre las siete figuras disponibles. La aleatoriedad asegura variedad en el juego, y el retorno de un puntero a la pieza permite que el juego maneje cada pieza independientemente. Facilita la integración con las funciones de creación y rotación de piezas.

Módulo Juego.

Como se comentó anteriormente, este módulo carga con la carga lógica más pesada del juego, usa las máscaras, el "bitwise", y los algoritmos que evalúan todas las condiciones que hacen posible el juego. Para ver la implementación completa, se puede remitir al código fuente hecho en Qt, por ahora, las funciones fundamentales:

```
Algoritmo: colision (tablero, pieza, alto, ancho, filaPieza, columnaPieza)
ENTRADA: tablero (apuntador a apuntador), pieza (arreglo 4x4), alto y
   ancho del tablero, filaPieza, columnaPieza
```

```
SALIDA: booleano (true si hay colisión, false si no)

PARA fila = 0 HASTA 3:

    PARA columna = 0 HASTA 3:

        SI pieza[fila] tiene bit activo en columna:

            filaTablero = filaPieza + fila

            columnaTablero = columnaPieza + columna

            // Verifica límites del tablero

            SI columnaTablero < 0 O columnaTablero >= ancho O filaTablero
   >= alto:

                RETORNAR true

            FIN SI

            // Solo revisar posiciones válidas dentro del tablero

            SI filaTablero >= 0:

                byteIndex = columnaTablero / 8

                bitIndex = columnaTablero % 8

                SI tablero[filaTablero][byteIndex] tiene bit activo en
   bitIndex:

                    RETORNAR true

                FIN SI

            FIN SI

        FIN SI

    FIN PARA

FIN PARA

RETORNAR false
```

```
FIN Algoritmo
```

Este algoritmo es esencial para controlar los movimientos de las piezas, evitando que se salgan del tablero o se superpongan a otras piezas. Garantiza que los movimientos sean válidos y seguros.

```
Algoritmo: moverIzquierda (tablero, pieza, alto, ancho, fila, columna)
ENTRADA: tablero, pieza, alto, ancho, fila y columna actual
SALIDA: booleano (true si la pieza se movió, false si no)
SI colision (tablero, pieza, alto, ancho, fila, columna - 1) ES false:
    columna = columna - 1
    RETORNAR true
SINO:
    RETORNAR false
FIN SI
FIN Algoritmo
```

Controla el movimiento horizontal hacia la izquierda. Se asegura de que la pieza no atraviese otras piezas ni los límites del tablero. De manera análoga se realizó el movimiento a la derecha.

```
Algoritmo: bajar (tablero, pieza, alto, ancho, fila, columna)
ENTRADA: tablero, pieza, alto, ancho, fila y columna actual
SALIDA: booleano (true si la pieza se movió, false si no)
SI colision (tablero, pieza, alto, ancho, fila + 1, columna) ES false:
    fila = fila + 1
    RETORNAR true
```

```
SINO:
    RETORNAR false
FIN SI
FIN Algoritmo
```

Controla el movimiento vertical de la pieza. Garantiza que la pieza solo avance hacia abajo si no hay colisiones, simulando la gravedad del juego y evitando errores en la colocación de bloques. En esta sección de pseudocódigo, se pudo observar la lógica más fuerte del código implementado, el corazón del proyecto y los algoritmos que mejor resumen la estructura de la solución implementada. El resto del código se remite a otras validaciones, funciones de interacción, de condiciones de barrera y demás. No está demás hacer énfasis nuevamente en remitirse al código fuente para revisar por completo el proyecto.

Apreciaciones Finales

Como parte final del informe, sería prudente acercarnos a la parte fuera del código, al inicio del proyecto y como se llega a esta implementación. Uno de los principales problemas de implementación iniciales, fue el manejo de memoria dinámica, aprender correctamente “bitwise” y las demás cosas que fueron usadas en esta; también, un gran problema en el desarrollo fue que la eficiencia y el manejo de bits no cambiara si la arquitectura de la máquina cambiara por cualquier razón, para esto, tomamos los enteros de ancho fijo (cpp reference, s.f.)También, como dificultad agregada, el hecho de separar interfaz gráfica de lógica representa un gran reto y deja una importante conclusión; lo gráfico tiene un gran costo operativo, al menos en estos niveles iniciales y es preciso ser muy cuidadoso con su manejo e implementación. Otros métodos interesantes a analizar son los usados respecto a la palabra reservada “cin” extraídas de (Geeksforgeeks, 2025).

Conclusiones

Retroalimentación De La Actividad Evaluativa.

El desafío I presentó ante nosotros un gran reto sin duda, es un problema complejo, que requiere de un tiempo prudente de análisis, conocimientos técnicos en c++ y sobre todo en las operaciones de tipo “bitwise”. Sin embargo, tenemos algunas apreciaciones, que, con el máximo respeto posible nos gustaría ilustrar de forma breve. Consideramos que el aprendizaje adquirido al realizar el proyecto es bueno, sin embargo, también consideramos que, al empezar a ver el lenguaje con un problema tan complejo, mucho del aprendizaje se vuelve más mecanización que entendimiento; vale la pena recalcar que no consideramos que el curso esté mal planteado, es más bien una consideración enfocada a los tipos de evaluación y se acerca más a nuestras propias limitaciones que al planteamiento del curso.

Otro punto interesante a recalcar, es la desconexión del laboratorio con el tema del desafío, aunque esto, es de manera más personal, por decirlo así, es más un deseo personal que una retroalimentación o una queja. Y es que nos parecería prudente, que, en tiempos de desafío, los laboratorios se conviertan en “reuniones” con asesores con otras perspectivas, que serían compañeros y profesores; sin embargo, entendemos que el curso debe continuar y hay temas por cubrir. Finalmente, siendo insistentes, recordamos que es algo más personal, y surge de nuestra necesidad de acompañamiento al enfrentarnos a un problema de alta complejidad. Entonces, para finalizar, si bien entendemos la complejidad del curso, las pautas puestas por el docente al iniciar el curso y la importancia de hacerlo así en cuánto a nuestro desarrollo personal y académico,

quisimos considerar estas apreciaciones a modo de retroalimentación del desafío I y lo que va del curso en general.

Uso De IA En El Proyecto.

Si bien una de las principales restricciones del programa es no usar IA generativa, consideramos que el uso de IA de manera no generativa es necesario, debido al permanente surgimiento de dudas; al iniciar el proyecto se presentó una necesidad casi permanente de asesoramiento, de ayuda en la búsqueda de errores y de retroalimentación general. Sabiendo aún que el profesor brinda asesorías, y que, a mediados del desafío encontramos monitor para el grupo y demás. Por cuestiones temporales, de cumplimiento de responsabilidades con otras materias y de trabajo, no encontramos otra alternativa para esta necesidad más que preguntar a la IA. Cabe aclarar que no desmeritamos los acompañamientos brindados por el curso, más bien resaltamos que no pudimos disponer siempre de estas ayudas y buscamos alternativas para poder resolver el desafío en el tiempo propuesto y de la manera más eficiente posible.

Reconocemos nuestra autoría de los códigos realizados, reconocemos también la ayuda constante de la IA como un “asesor” para el proyecto y estamos dispuestos a enfrentar las consecuencias posteriores. Quisimos reconocer esto porque consideramos que es honesto no solo con el profesor, sino también con nuestro proceso de aprendizaje y en como implementamos la solución. Consideramos pertinente evaluar para desafíos y evaluaciones futuras, la inclusión de la IA como medio precursor de retroalimentación y sobre todo de ayuda en el planteamiento lógico y en la orientación de las posibles ideas de solución.

Conclusión Final.

Este proyecto fue una gran ayuda para el desarrollo de la lógica enfocada a la resolución de problemas, de cómo afrontamos una situación problema y sobre todo del manejo de c++ y del IDE Qt, entre otros aspectos que cubre el desafío. En general, el desafío I de Informática II presenta una gran dificultad para nosotros como estudiantes, sin embargo, ayuda a pulir estos conceptos básicos que el profesor nos ha avisado que serán fundamentales para la conclusión del curso. El proceso de desarrollo, aunque estresante fue satisfactorio y nos deja con lo que creemos que será una gran entrega.

Referencias

cpp reference. (s.f.). Obtenido de https://en.cppreference.com/w/cpp/types/integer.html

Geeksforgeeks. (11 de Diciembre de 2023). Obtenido de https://www.geeksforgeeks.org/cpp/bitmasking-in-cpp/

Geeksforgeeks. (23 de Julio de 2025). Obtenido de https://www.geeksforgeeks.org/cpp/cin-in-c/

Granada, U. d. (s.f.). *Universidad de Granada*. Obtenido de https://ccia.ugr.es/~jfv/ed1/c++/cdrom4/paginaWeb/stl.htm

MSMK | University College. (30 de Mayo de 2025). Obtenido de https://msmk.university/que-es-camel-case/

UdeA, F. d. (s.f.). *Metodología* . Medellín: Ude@. Obtenido de chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://udearroba.udea.edu.co/internos/pluginfile.php/3090794/mod_resource/content/1/inforII-metodologia.pdf